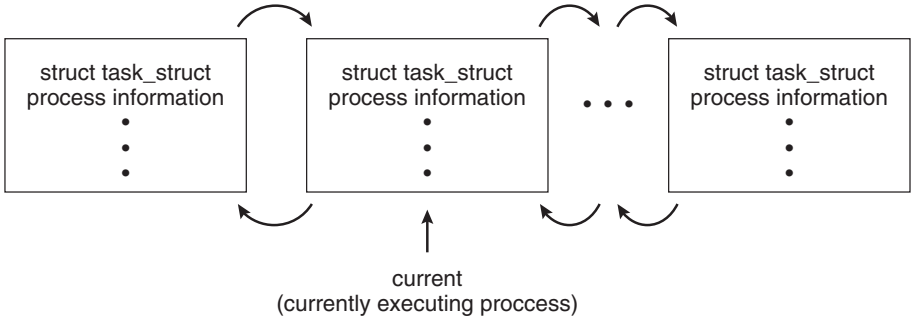


برای این که نمایشی از چگونگی دستکاری یکی از فیلدهای رکورد task_struct توسط هسته برای پردازه‌ای مشخص را نشان بدهیم، فرض کنید که سیستم می‌خواهد حالت پردازه‌ی در حال اجرای فعلی را به مقدار new_state تغییر بدهد. اگر current اشاره‌گری باشد که به پردازه‌ی در حال اجرای فعلی اشاره می‌کند، حالت پردازه با دستور زیر تغییر می‌یابد:

```
current->state = new_state;
```

۳-۱-۴ نخ‌ها

از مدل پردازه‌ای که تاکنون بحث شد، چنین بر می‌آید که پردازه، برنامه‌ای است که فقط یک **نخ**[1] اجرایی منفرد را به انجام می‌رساند. برای مثال، زمانی که پردازه‌ای برنامه‌ی واژه‌پرداز را اجرا می‌کند، نخ منفردی از دستورالعمل‌ها اجرا می‌شوند. این تک‌نخ کنترل، به پردازه امکان می‌دهد که فقط یک وظیفه را در هر لحظه انجام بدهد. بنابراین، کاربر نمی‌تواند همزمان کاراکترها را تایپ کرده و غلط‌یاب املاء[2] را اجرا کند. اکثر سیستم‌عامل‌های مدرن، مفهوم پردازه را تعمیم داده و اجازه می‌دهند که یک پردازه چندین نخ اجرایی داشته باشد و از این رو بتواند در هر لحظه بیش از یک وظیفه را انجام بدهد. این ویژگی مخصوصاً در سیستم‌های چندهسته‌ای سودآور است که نخ‌های متعددی می‌توانند به صورت موازی اجرا شوند. برای مثال، یک واژه‌پرداز چندنخی، یک نخ را به مدیریت ورودی کاربر می‌گمارد در حالی که نخ دیگری غلط‌یاب املاء را اجرا می‌کند. در سیستم‌هایی که از نخ‌ها پشتیبانی می‌کنند، PCB توسعه داده می‌شود تا اطلاعات هر یک از نخ‌ها را نیز لحاظ کند. تغییرات دیگری نیز جهت پشتیبانی نخ‌ها لازم است در کل سیستم صورت بگیرد. فصل ۴، نخ‌ها را با جزئیات بیشتری بررسی می‌کند.

۳-۲ زمانبندی پردازه

هدف چندبرنامگی، داشتن چندین پردازه‌ی در حال اجرا در تمامی زمانهاست تا بهره‌وری پردازنده بیشینه شود. هدف اشتراک‌زمانی این است که یک هسته‌ی پردازشی چنان مکرر مابین

1. Thread
2. Spell checker

پردازه‌ها رد و بدل شود که کاربران با هر برنامه‌ی در حال اجرا تعامل داشته باشند. برای حصول این اهداف، **زمانبند پردازه** پردازه‌ی آماده‌ای را (احتمالاً از میان مجموعه‌ی پردازه‌های آماده‌ی متعدد) برای اجرای برنامه بر روی یک هسته انتخاب می‌کند. هر هسته‌ی CPU می‌تواند در هر لحظه یک پردازه را اجرا کند. در یک سیستم با یک هسته‌ی CPU منفرد، هرگز بیش از یک پردازه‌ی در حال اجرا وجود نخواهد داشت، در حالی که یک سیستم چندهسته‌ای می‌تواند پردازه‌های متعددی را در یک لحظه اجرا کند. اگر تعداد پردازه‌ها بیشتر از تعداد هسته‌ها باشند، پردازه‌های اضافی ناگزیر منتظر خواهند ماند تا این که یک هسته آزاد شده و دومرتبه زمان‌بندی شود. تعداد پردازه‌های جاری در داخل حافظه، تحت عنوان **درجه‌ی چندبرنامگی** شناخته می‌شود.

موازنه‌ی اهداف چندبرنامگی و اشتراک‌زمانی مستلزم در نظر گرفتن رفتار کلی یک پردازه است. در حالت کلی، اکثر پردازه‌ها می‌توانند به‌صورت **پاپیچ I/O**[1] و یا **پاپیچ CPU**[2] بیان شوند. یک پردازه‌ی پاپیچ I/O پردازه‌ای است که بیشتر اوقات خود را به انجام I/O صرف می‌کند تا انجام محاسبات. در مقابل، یک پردازه‌ی پاپیچ CPU به‌ندرت درخواست I/O صادر می‌کند و بیشتر اوقات خود را به‌انجام محاسبات صرف می‌کند.

۳-۲-۱ صف‌های زمانبندی

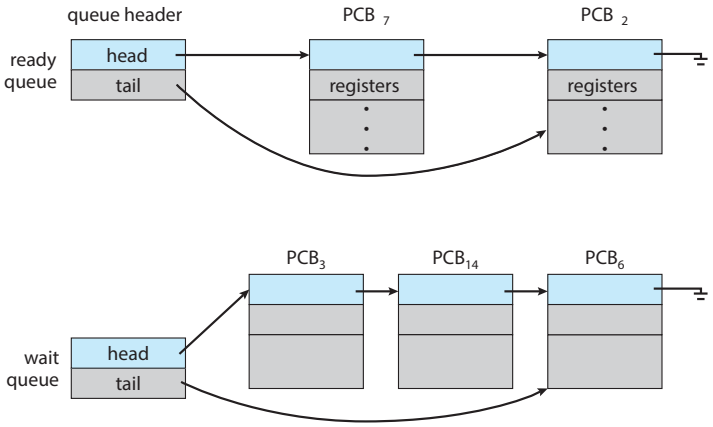
پردازه‌ها با ورود به سیستم در **صف آماده** گذاشته می‌شوند که در آنجا آماده بوده و به‌انتظار اجرا بر روی یک هسته‌ی پردازنده می‌باشند. این صف به طور کلی به صورت یک لیست پیوندی ذخیره می‌شود؛ سرآیند صف آماده اشاره‌گرهایی به اولین و آخرین PCB در لیست دارد، و هر PCB یک فیلد اشاره‌گر دارد که به PCB بعدی صف آماده اشاره می‌کند.

سیستم، صف‌های دیگری را هم شامل است. زمانی که پردازه‌ای، یک هسته‌ی پردازنده را در اختیار می‌گیرد، برای مدت زمانی اجرا شده و در نهایت، خاتمه یافته، وقفه دریافت می‌کند یا منتظر رخداد رویداد خاصی همچون تکمیل یک درخواست I/O می‌ماند. فرض کنید پردازه یک درخواست I/O برای دستگاهی همچون دیسک انجام می‌دهد. از آنجایی که دستگاه‌ها خیلی کندتر از پردازنده‌ها اجرا می‌شوند، پردازه ناگزیر منتظر I/O خواهد ماند تا در دسترس قرار بگیرد. پردازه‌هایی که منتظر رخداد یک رویداد خاصی هستند—مثلاً تکمیل I/O—در یک **صف انتظار** گذاشته می‌شوند (شکل ۳-۴).

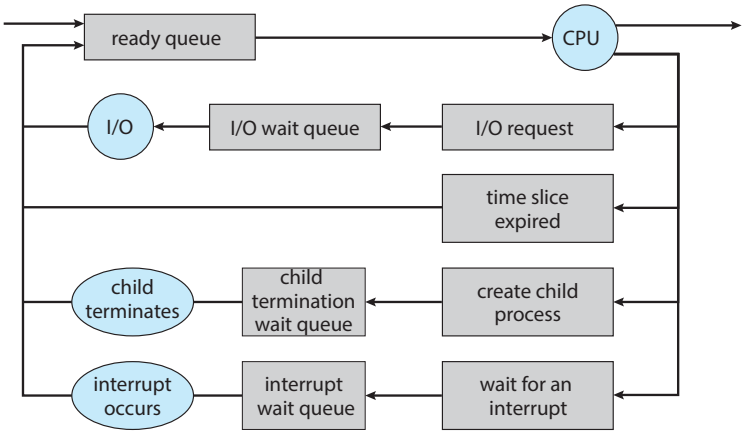
یک نمایش مرسوم از زمانبندی پردازه، **دیاگرام صف‌بندی**[3] است که نمونه‌ای از آن در شکل ۳-۵ نشان داده می‌شود. دو نوع صف وجود دارند که عبارتند از: صف آماده و مجموعه‌ای از صف‌های انتظار. دایره‌ها منابعی را نشان می‌دهند که به صف‌ها سرویس می‌دهند و پیکان‌ها، جریان پردازه‌ها را در سیستم مشخص می‌کنند.

1. I/O bound
2. CPU bound
3. Queuing diagram

پردازه‌ی جدید نخست در صف آماده گذاشته می‌شود. در آنجا منتظر می‌ماند تا این که برای اجرا انتخاب شده یا **اعزام شود**[1]. همین که پردازه یک هسته‌ی پردازنده را در اختیار گرفت و اجرای خود را آغاز کرد، یکی از چندین رویداد زیر می‌تواند اتفاق بیفتد:



شکل ۳-۴ صف آماده و صف‌های انتظار.



شکل ۳-۵ نمایش نمودار صف‌بندی زمان‌بندی پردازه.

- پردازه می‌تواند یک درخواست I/O صادر کرده و سپس در یک صف انتظار I/O گذاشته شود.
- پردازه می‌تواند یک پردازه‌ی فرزند جدید ایجاد کرده و سپس در یک صف انتظار گذاشته شود، در حالی که منتظر ختم فرزند می‌باشد.

1. Dispatch

- پردازه ممکن است پیرو یک وقفه، یا انقضای سهم زمانیش ناگزیر از هسته کنار گذاشته شده و دومرتبه در صف آماده قرار داده شود.

در دو مورد اول، پردازه بالاخره از حالت انتظار خارج شده و به حالت آماده می‌رسد و سپس دومرتبه در صف آماده گذاشته می‌شود. هر پردازه این چرخه را ادامه می‌دهد تا این که در نهایت خاتمه بپذیرد، در اینجاست که از تمامی صف‌ها حذف شده و PCB و منابع در اختیارش پس‌گرفته می‌شوند.

۳-۲-۲ زمانبندی پردازنده

یک پردازه در طول مدت حیات خود مابین صف آماده و صف‌های انتظار مختلفی جابه‌جا می‌شود. نقش **زمان‌بند پردازنده** این است که یکی را از میان پردازه‌هایی انتخاب کند که در صف آماده هستند و یک هسته‌ی پردازنده را به آن تخصیص دهد. زمان‌بند پردازنده بایستی مکرراً پردازه‌ی جدیدی را برای پردازنده انتخاب کند. یک پردازه‌ی پاپیچ I/O ممکن است تنها به مدت چند میلی‌ثانیه اجرا شود، قبل از این که منتظر یک درخواست I/O بماند. اگرچه یک پردازه‌ی پاپیچ پردازنده، یک هسته‌ی پردازنده را برای فواصل طولانی‌تری لازم خواهد داشت، زمان‌بند بعید است که هسته را به یک پردازه برای یک دوره‌ی بیشتری اعطا کند. در عوض، احتمالاً جوری طراحی می‌شود تا با اجبار، پردازنده را از یک پردازه گرفته و برای اجرای پردازه‌ای دیگر زمان‌بندی کند. بنابراین، زمان‌بند پردازنده حداقل یک‌بار در هر ۱۰۰ میلی‌ثانیه اجرا می‌شود، هرچند که نوعاً با دفعات بسیار بیشتری هم اجرا دارد.

برخی سیستم‌عامل‌ها یک شکل بینابینی از زمان‌بندی دارند که به **مبادله**[1] معروف است که ایده‌ی کلیدی آن این است که بعضاً بهتر است یک پردازه از حافظه (و از رقابت فعال برای پردازنده) حذف شده و لذا درجه‌ی چندبرنامگی کاهش پیدا کند. پردازه می‌تواند بعداً دومرتبه به داخل حافظه آورده شده و اجرایش را از همان جایی که مانده بود تداوم ببخشد. این طرح از این نظر به **مبادله** معروف است که یک پردازه می‌تواند از حافظه به دیسک «به بیرون مبادله شود» که در آنجا وضعیت فعلیش ذخیره شده و بعداً دوباره از دیسک به حافظه «به داخل مبادله شود» که در آنجا وضعیتش بازیابی می‌شود. مبادله نوعاً فقط زمانی ضروری است که حافظه بیش از حد اشغال شده است و بایستی جای خالی پیدا شود. مبادله در فصل ۹ مورد بحث قرار می‌گیرد.

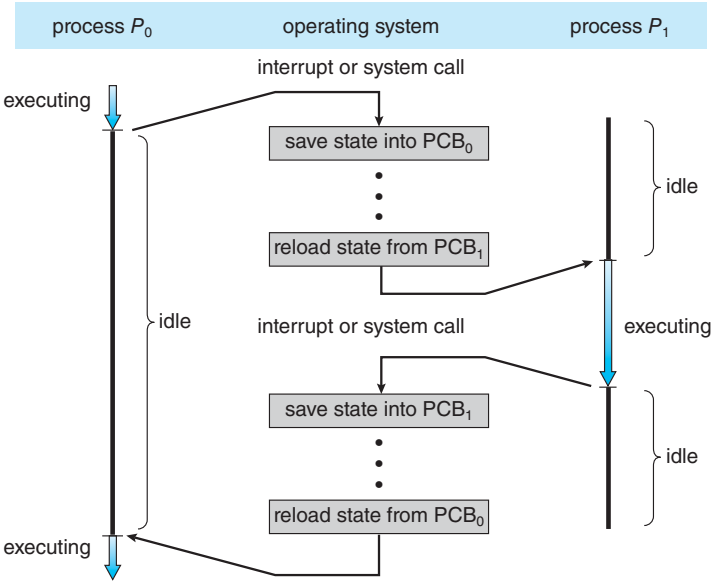
۳-۲-۳ برگردان متن

همان‌گونه که در بخش ۱-۲-۱ متذکر شدیم، وقفه‌ها باعث می‌شوند سیستم‌عامل یک هسته‌ی پردازنده را از کار فعلی خود برگردانده و روالی را از هسته اجرا کند. چنین عملیاتی به دفعات در سیستم‌های همه‌منظوره اتفاق می‌افتند. زمانی که وقفه‌ای رخ می‌دهد سیستم لازم دارد که متن جاری پردازه‌ی در حال اجرا در هسته‌ی پردازنده را ذخیره نماید تا بتواند آن متن را بعد از انجام پردازش خود بازیابی کند و در اصل، پردازه را معلق کرده و دوباره از سر بگیرد. متن

1. Swapping

در **PCB** پردازه نشان داده می‌شود که شامل مقادیر ثبات‌های پردازنده، حالت پردازه (شکل ۳-۲ را ببینید) و اطلاعات مدیریت حافظه می‌باشد. به‌طور کلی، ما یک ذخیره‌ی حالت برای حالت فعلی هسته‌ی پردازنده انجام می‌دهیم، فارغ از این که در مد هسته یا کاربر باشد، و سپس برای از سرگیری عملیات، یک بازیابی حالت خواهیم داشت.

لازمه‌ی انتقال هسته‌ی پردازنده به پردازه‌ای دیگر، انجام یک ذخیره‌ی حالت برای پردازه‌ی فعلی و یک بازیابی حالت برای پردازه‌ای دیگر است. این کار به **برگردان متن**[1] معروف است و در شکل ۳-۶ نمایش داده می‌شود. زمانی که برگردان متنی اتفاق می‌افتد، هسته متن پردازه‌ی پیشین را در بلاک کنترل پردازه ذخیره کرده و متن ذخیره شده‌ی پردازه‌ی جدید را که برای اجرا زمانبندی شده است بار می‌کند. زمان برگردان متن، سربار محض است، چرا که سیستم در زمان رد و بدل شدن پردازنده، هیچ‌گونه کار مفیدی انجام نمی‌دهد. سرعت انجام برگردان متن بسته به سرعت حافظه، تعداد ثبات‌هایی که بایستی کپی شوند و وجود دستورالعمل‌های خاص، (همچون دستورالعمل منفردی که بتواند تمامی ثبات‌ها را بارگذاری یا ذخیره نماید) از ماشینی به ماشین دیگر متفاوت است. سرعت نوعی آن در حد چند میکروثانیه است.



شکل ۳-۶ نموداری که برگردان متن را از یک پردازه به پردازه‌ی دیگر نشان می‌دهد.

زمان‌های برگردان متن، وابستگی زیادی به پشتیبانی سخت‌افزاری دارد. برای نمونه، برخی پردازنده‌ها، چندین مجموعه ثبات فراهم می‌کنند. برگردان متنی که در اینجا صورت می‌گیرد به سادگی تغییر اشاره‌گر به مجموعه ثبات جاری می‌باشد. البته، اگر تعداد

1. Context switch

پردازه‌های فعال بیش از تعداد مجموعه‌ی ثبات‌ها باشد، سیستم همانند حالت قبل، داده‌های ثبات‌ها را به حافظه کپی کرده و دوباره از حافظه بار می‌کند. هر اندازه سیستم‌عامل پیچیده‌تر باشد، بایستی در طی برگردان متن نیز کار بیشتری انجام شود. همان‌گونه که در فصل ۹ خواهیم دید، تکنیک‌های مدیریت حافظه‌ی پیشرفته ممکن است لازم داشته باشند که در انجام هر یک از برگردان‌های متن، داده‌های اضافه‌تری منتقل شوند. برای نمونه، در حالی که فضای آدرس کار بعدی برای استفاده‌اش آماده می‌شود، بایستی فضای آدرس پردازه‌ی فعلی حفظ شود. این که فضای آدرس چگونه حفظ می‌شود و چه میزان کار برای حفظ آن لازم است، به روش مدیریت حافظه‌ی سیستم‌عامل بستگی دارد.

چندوظیفگی در سیستم‌های موبایل

به دلیل محدودیت‌هایی که بر روی دستگاه‌های موبایل وجود دارد، نسخه‌های اولیه‌ی iOS چندوظیفگی برنامه‌های کاربر را فراهم نمی‌کرد؛ تنها یک برنامه می‌توانست در پیش‌زمینه اجرا شود، در حالی که سایر برنامه‌های کاربر همگی معلق می‌شدند. وظایف سیستم‌عامل چندوظیفه بودند چون توسط اپل نوشته می‌شدند و رفتار خوبی داشتند. با این وجود، با شروع از iOS 4، اَپل شکل محدودی از چندوظیفگی را برای برنامه‌های کاربر فراهم کرد، لذا یک برنامه‌ی پیش‌زمینه‌ی منفرد اجازه دارد که به صورت همروند با برنامه‌های پس‌زمنیه‌ی متعدد اجرا شود. (در دستگاه موبایل، برنامه‌ی **پیش‌زمینه** برنامه‌ای است که فعلاً باز است و در نمایشگر دیده می‌شود. برنامه‌ی **پس‌زمینه** در حافظه باقی می‌ماند، اما نمی‌تواند صفحه‌ی نمایشگر را در اختیار بگیرد.) API برنامه‌نویسی iOS 4 پشتیبانی چندوظیفگی را فراهم کرد، لذا امکان اجرا را به یک پردازه در پس‌زمینه بی آنکه تعلیق شود، فراهم می‌کند. با این وجود، محدودیت داشته و تنها برای تعداد معدودی از انواع برنامه‌ها در دسترس می‌باشد. همچنان‌که سخت‌افزار دستگاه‌های موبایل به عرضه‌ی ظرفیت‌های بزرگ‌تر حافظه، هسته‌های پردازشی متعدد، و طول عمر باتری بیشتر روی آورد، نسخه‌های بعدی iOS شروع کردند به پشتیانی کارکرد غنی‌تری برای چندوظیفگی با محدودیت‌های کم‌تر. برای مثال، صفحه‌نمایش بزرگ‌تر در تبلت‌های iPad اجازه داد تا دو برنامه‌ی پیش‌زمینه همزمان اجرا شود، تکنیکی که به **صفحه‌نمایش شکسته**[1] معروف است.

اندروید از زمان شروعش از چندوظیفگی پشتیبانی کرده و روی انواع برنامه‌هایی که می‌توانند در پس‌زمینه اجرا شوند محدودیتی نمی‌گذارد. اگر برنامه‌ای نیازمند پردازش در پس‌زمینه است، برنامه بایستی از یک سرویس، که یک جزء برنامه‌ی مجزاست که از طرف پردازه‌ی پس‌زمینه اجرا می‌شود، استفاده کند. برنامه‌ی صوتی در جریان را در نظر بگیرید: اگر برنامه به پس‌زمینه منتقل شود، این سرویس از جانب برنامه‌ی پس‌زمینه، ارسال داده‌های صوتی را به درایور دستگاه صوتی ادامه می‌دهد. در حقیقت، حتی در صورتی که برنامه‌ی پس‌زمینه به حالت تعلیق هم در آید، اجرای سرویس ادامه خواهد داشت. سرویس‌ها رابط کاربری نداشته و ردپای حافظه‌ی کوچکی دارند، از این رو یک تکنیک کارآمد برای چندوظیفگی در یک محیط موبایل فراهم می‌کنند.

1. Split-screen